Natural Language Processing

A.Schutz 2025

Déchiffrer le langage humain avec l'IA

Définition: Le Traitement du Langage Naturel (NLP) est une branche de l'IA qui vise à permettre aux ordinateurs de comprendre, d'interpréter et de générer le langage humain.

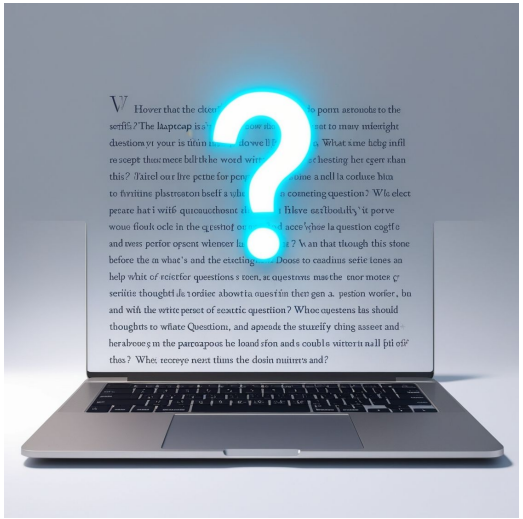
Applications: Analyse de sentiments, traduction automatique, chatbots, assistants virtuels, résumé de texte, etc.



Pourquoi transformer le texte ?

Problème: Les algorithmes de Machine Learning et les réseaux de neurones travaillent avec des nombres, pas avec du texte brut.

Solution: Transformer le texte en une représentation numérique que les machines peuvent comprendre.



L'arsenal du NLP en Python

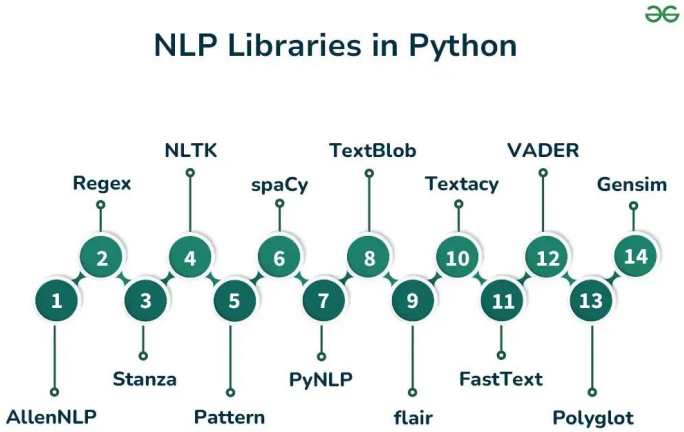
SKLearn: Bibliothèque de Machine Learning généraliste incluant des outils de vectorisation de texte (TF-IDF, CountVectorizer) pour le NLP.

NLTK (Natural Language Toolkit): Bibliothèque complète pour le NLP, avec des outils pour la tokenisation, le stemming, la lemmatisation, etc.

spaCy: Bibliothèque rapide et efficace pour le NLP, avec des modèles pré-entraînés pour différentes langues.

Gensim: Bibliothèque spécialisée dans la modélisation de sujets et la similarité de documents.

Transformers (Hugging Face): Bibliothèque pour utiliser des modèles de langage pré-entraînés (ex: BERT, GPT).





https://www.geeksforgeeks.org/nlp-libraries-in-python/

Appliquer le Machine Learning au langage

Classification: Classer des textes en catégories (ex: analyse de sentiments, détection de spam).

Clustering: Regrouper des textes similaires (ex: analyse de sujets, segmentation de clients).

RNN: Adaptés aux données séquentielles comme le texte.

LSTM et **GRU**: Variantes de RNN qui peuvent gérer des dépendances à long terme dans le texte.

Transformers: Architecture plus récente et performante pour le NLP, avec des modèles comme BERT et GPT.

Importer les outils de NLTK (venv)

```
# Obtenir le chemin absolu du répertoire de données NLTK
nltk_data_dir = os.path.abspath('./.venv/nltk_data')

# Définir la variable d'environnement NLTK_DATA
os.environ['NLTK_DATA'] = nltk_data_dir

# Télécharger les ressources NLTK
nltk.download('punkt', download_dir=nltk_data_dir)
nltk.download('stopwords', download_dir=nltk_data_dir)
nltk.download('wordnet', download_dir=nltk_data_dir)
nltk.download('punkt_tab', download_dir=nltk_data_dir)
```

tester le tokenizer

```
# On test le tokenizer
text = "This is an example sentence."
tokens = nltk.word_tokenize(text)
print(tokens)
```

```
['This', 'is', 'an', 'example', 'sentence', '.']
```

Transformations du texte
Veille technologique avec code

Tokenisation: Découper le texte en unités individuelles (tokens), comme des mots ou des phrases.

Suppression des stop words: Supprimer les mots courants (articles, prépositions, etc.) qui n'apportent pas beaucoup d'information.

Stemming/Lemmatisation: Réduire les mots à leur racine (stemming) ou à leur forme canonique (lemmatisation).

Vectorisation: Transformer les tokens en vecteurs numériques (ex: Bag-of-Words, TF-IDF, Word2Vec).

Veille Techno !

et en plus

N-Grams

Corpus

lem and stem

Stemming : Le stemming est un processus heuristique qui consiste à supprimer les suffixes des mots pour les ramener à leur racine. Il se base souvent sur des règles simples et ne prend pas en compte le contexte du mot. Pour simplifier le processus et améliorer l'efficacité, le stemming met généralement tous les mots en *minuscules* avant d'appliquer les règles de suppression des suffixes.

Lemmatisation : La lemmatisation est un processus plus sophistiqué qui utilise un dictionnaire et une analyse morphologique pour ramener les mots à leur forme canonique (lemme). Le lemme est la forme du mot que l'on trouve dans un dictionnaire.

WordCloud

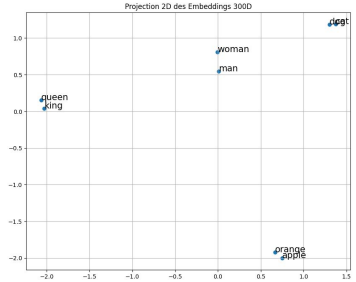
```
from wordcloud import WordCloud, STOPWORDS
# Créer un ensemble de stop words (vide dans ce cas)
stopwords = set()

# Générer le WordCloud
wordcloud = WordCloud(width = 800, height = 800,
                background_color ='white',
                stopwords = stopwords,
                min_font_size = 10).generate_from_frequencies(frequences)

# Afficher le WordCloud
plt.figure(figsize = (8, 8), facecolor = None)
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad = 0)
```



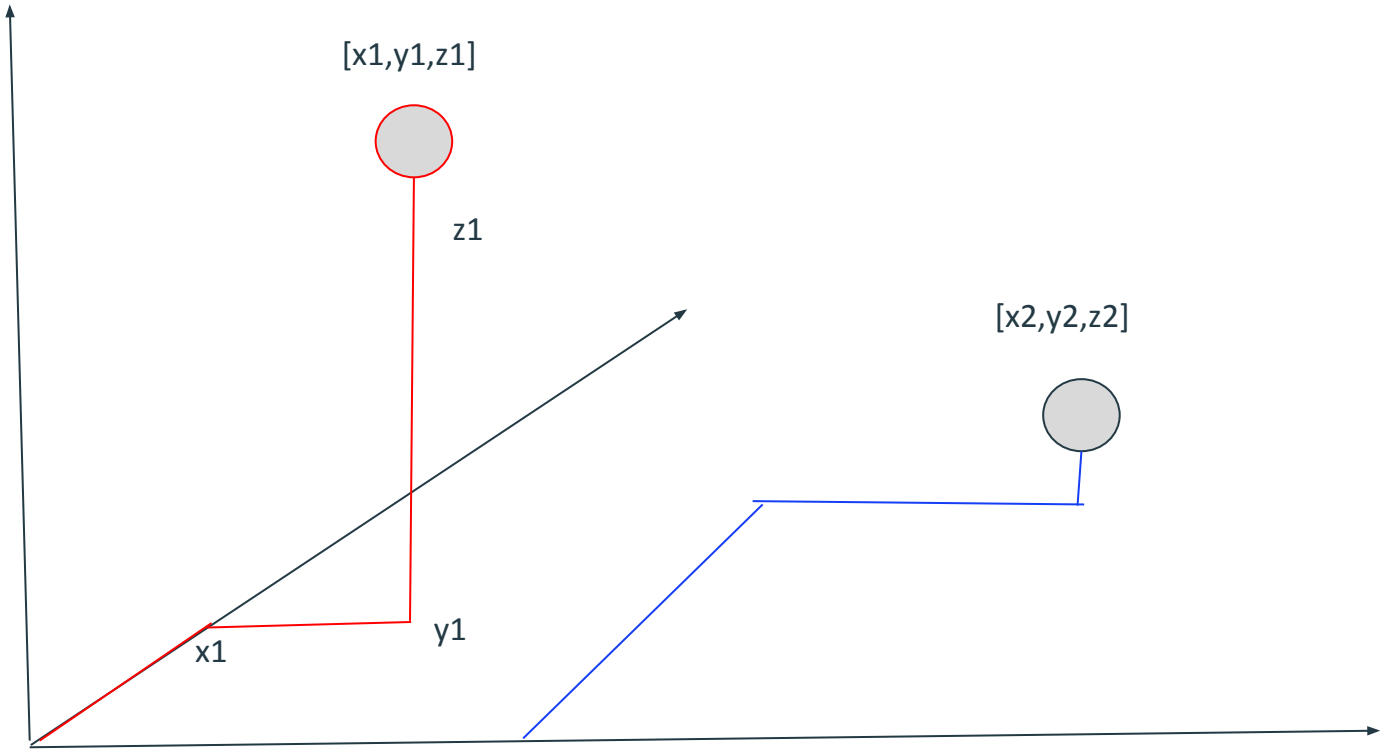
L'embedding

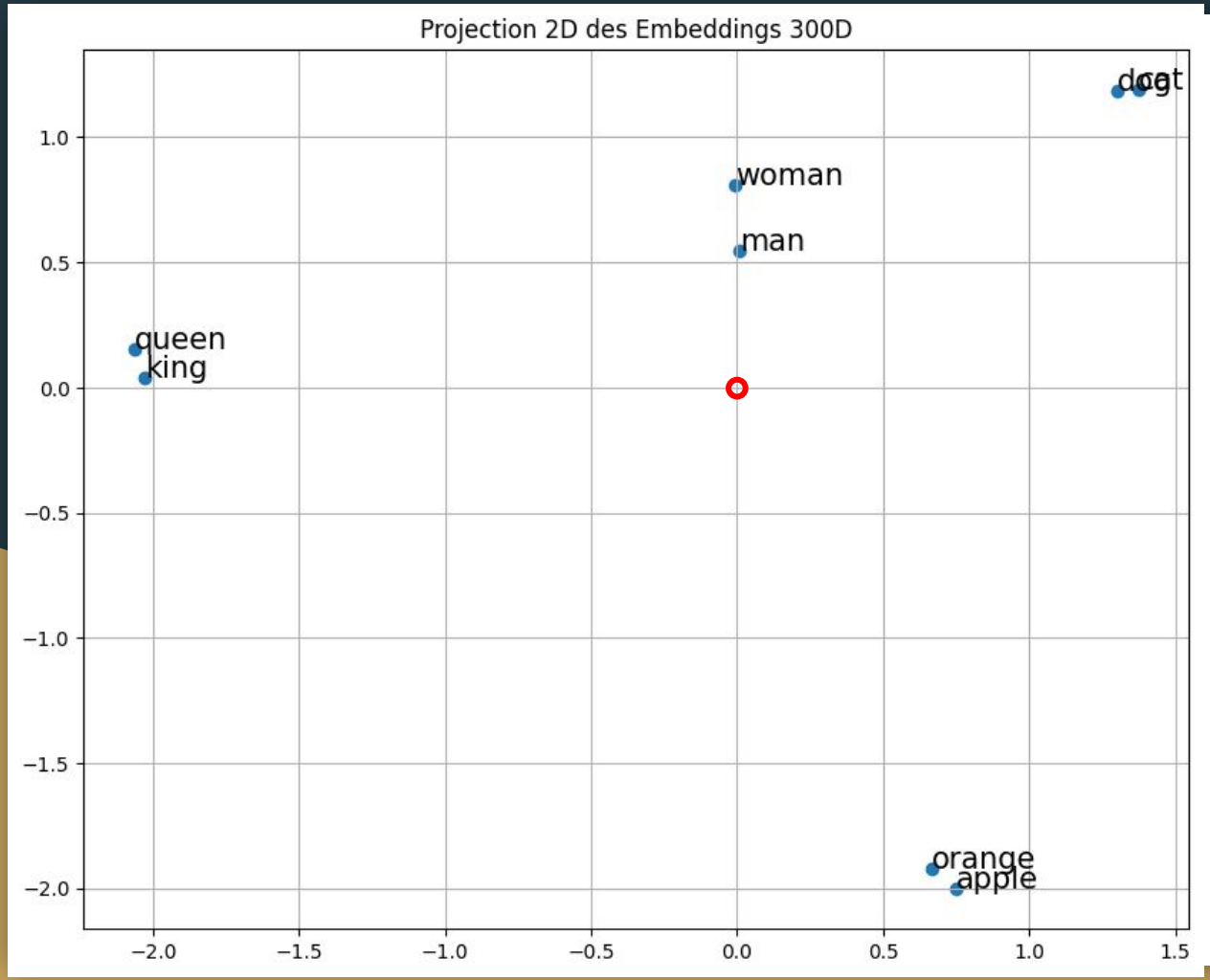


L'embedding est une technique qui transforme chaque mot en une liste de nombres (un vecteur) où la position dans l'espace reflète le sens : plus deux mots ont un sens proche (comme "bateau" et "navire"), plus ils seront géographiquement proches dans cet espace.

Contrairement au TF-IDF qui ne voit que l'orthographe, l'embedding apprend la relation entre les mots en observant leur voisinage dans des millions de textes : il comprend ainsi que "Paris" est à la "France" ce que "Tokyo" est au "Japon".

Contrairement au NLP version machine learning à l’ancienne et dans un contexte de deep learning on conserve le maximum d’information et on utilise juste la tokenisation.


[x1,y1,z1]
z1
[x2,y2,z2]
y1
x1



$$Similarity(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

2013 (Google-**Tomas Mikolov)** : **Word2Vec**.

2017 (L'architecture **Transformer) :** *"Attention Is All You Need"*

2018 : **BERT**